

Advanced Standard Mode: Safe-to-Use Technique Combinations for Transferring Solver State Across Related Verifications

VHAGaR / neta-s-paper — Internal technical note

Abstract

Advanced standard (`advstd`) mode is an execution mode of the VHAGaR global-robustness verifier that computes the global robustness bound $\delta_{N_2}^*$ for a target network N_2 by *reusing solver state* from a prior solve of a structurally similar network N_1 .

This note documents only the **safe-to-use** technique combinations — those empirically validated to produce stable speedups with no severe slowdowns (worst-case slowdown $> 0.8\times$, no stable-lose seed). Out of the full combinatorial space, 17 combinations satisfy this criterion. They use four independent techniques (Sections 4.1–4.4), all of which preserve the optimal global-robustness bound δ_{exact} . Section 6 gives the complete algorithm for every safe combination.

1 Motivation and Setting

Let N_1 and N_2 be two neural networks with the same architecture — same layer sizes, same ReLU structure, same input domain — but different parameter values. A typical instance is $N_2 = N_1$ after one additional SGD epoch.

The global robustness verification problem, as formulated in VHAGaR, asks: for a given source class c_s , target class c_t , perturbation type P , and bound ε , what is the smallest δ such that no input classified with confidence at least δ as c_s can be perturbed within $\mathcal{I}_\varepsilon(\cdot)$ to be classified as c_t ? This is encoded as a mixed-integer program (MIP) and solved with Gurobi.

When N_1 and N_2 are structurally identical but numerically close, the two MIPs have identical variable names, identical constraint structure (only coefficients differ), and similar solution geometry. Advanced standard mode exploits this similarity through a set of concrete, optional techniques.

2 Two-Phase Execution

Advanced standard mode runs in two phases, possibly in separate processes:

Phase 1 — N_1 solve

For each (c_s, c_t) class pair, solve the standard-mode MIP for N_1 to either optimality or a user-specified timeout. Extract:

- The full primal solution $(\text{var}_i, \text{val}_i)_{i=1}^{|V|}$.
- The per-neuron interval bounds $(l_{m,k}, u_{m,k})$ on all split ReLUs (the `layers_info` dictionary).

Serialise to disk. The resulting `n1_state` directory is a self-contained snapshot of “everything we learned while solving N_1 ”.

Phase 2 — N_2 sweep

For each (c_s, c_t) pair, build the standard-mode MIP for N_2 , load the N_1 state, apply each enabled technique, and call `optimize!`. Phase 2 never re-solves N_1 .

3 Background: ReLU Encoding in the MIP

The VHAGaR MIP encodes each ReLU neuron $z^+ = \max(0, \hat{z})$ using a *big-M formulation* with a binary variable $a \in \{0, 1\}$. Given pre-activation bounds $\hat{z} \in [\underline{z}, \bar{z}]$ (where $\underline{z} < 0 < \bar{z}$ for a “split” neuron), the encoding introduces:

$$z^+ \geq 0, \tag{1}$$

$$z^+ \geq \hat{z}, \tag{2}$$

$$z^+ \leq \hat{z} - \underline{z}(1 - a), \tag{3}$$

$$z^+ \leq \bar{z}a. \tag{4}$$

This is *exact*: when $a = 1$ (neuron active), constraints force $z^+ = \hat{z}$; when $a = 0$ (neuron inactive), they force $z^+ = 0$. The binary a is what makes the problem a *mixed-integer* program — solving over all possible $\{0, 1\}$ assignments is what makes verification NP-hard.

The LP relaxation of ReLU. If we *drop* the binary a and keep only the three constraints that do not involve a , we get the *triangle LP relaxation*:

$$z^+ \geq 0, \tag{5}$$

$$z^+ \geq \hat{z}, \tag{6}$$

$$z^+ \leq \frac{\bar{z}}{\bar{z} - \underline{z}} (\hat{z} - \underline{z}). \tag{7}$$

This is a *convex relaxation*: it allows z^+ to take any value inside a triangle in the $(\hat{z}, z^+)$ plane, not just the two exact ReLU branches. There is no binary variable, so the problem becomes a linear program (LP) — much faster to solve, but the feasible set is *larger* than the exact one. Any solution to the exact encoding also satisfies the LP relaxation, but not vice versa.

The LP relaxation is used in two different contexts in this document:

1. **As a building block** inside Technique 4’s N_1 Probe (Source C): a full LP model with triangle-relaxed ReLUs is solved to compute tight per-neuron bounds.
2. **As a replacement** in Technique 6: selected binary variables in the final N_2 MIP are replaced with triangle relaxations to reduce the integer search space.

4 Techniques Used in Safe Combinations

The safe combinations use exactly four techniques. Technique 1 (MIP Start) and Technique 3 (LP Basis), which were part of the original advstd toolchain, are **not** used in any safe combination and are therefore omitted from this document.

Correctness invariant

Let $\mathcal{F}_2$ denote the feasible set of N_2 ’s standard-mode MIP and let

$$\delta_{\text{exact}} \triangleq \delta_{N_2}^* = \max_{x \in \mathcal{F}_2} \text{obj}_{N_2}(x).$$

For every subset of techniques, the optimal value reported by the solver when it terminates at OPTIMAL is exactly δ_{exact} . The techniques only change the *path* Gurobi takes through the branch-and-bound tree, not the set of feasible solutions or the objective function.

4.1 Technique 2 — Branch Priorities (bp=bounds)

Flag: `--adv_std_branch_priorities` $\in \{\text{off}, \text{bounds}\}$. **Gurobi attribute:** `BranchPriority`.

Idea. For each binary $a_{m,k}$ in N_2 , look up N_1 's final interval bounds $(l_{m,k}, u_{m,k})$ and compute the gap $g_{m,k} = |u_{m,k} - l_{m,k}|$. Assign a three-level priority:

$$\text{pri}(m, k) = \begin{cases} 10 & \text{if } g_{m,k} < 1 \\ 5 & \text{if } 1 \leq g_{m,k} < 5 \\ 1 & \text{otherwise.} \end{cases}$$

The intuition is that tight-bound neurons are hard-contested neurons, so N_2 should branch on them first.

Soundness. `BranchPriority` only changes the order in which the branch-and-bound tree is explored. No constraint is added: $\mathcal{F}_2^{\{2\}} = \mathcal{F}_2$ and δ_{exact} is preserved.

4.2 Technique 4 — Bound Tightening (boundTight)

Flag: `--adv_std_bound_tightening` (always true in safe combos). **Model modification:** tightens neuron bounds before the N_2 MIP is built.

This technique runs *before* the MIP is constructed. Its output — tighter per-neuron intervals — feeds into the MIP encoding itself (smaller big- M constants, fewer binary variables). The other techniques (branch priorities, variable hints, triangle relaxation) operate *on* the already-built MIP.

Technique 4 is independent of the *perturbed interval constraints* that are added to the MIP after construction (see the note at the end of this subsection).

Goal. For each ReLU neuron (m, k) in N_2 , compute a sound interval $[d_{m,k}^\downarrow, d_{m,k}^\uparrow]$ on the *weight-drift difference* $\hat{z}_{m,k}^{N_2} - \hat{z}_{m,k}^{N_1}$ (how much the pre-activation changed due to N_2 having different weights than N_1). Once we have this interval, we combine it with N_1 's known bounds from Phase 1 to get N_2 bounds:

$$l_{m,k}^{N_2} \geq l_{m,k}^{N_1} + d_{m,k}^\downarrow, \quad u_{m,k}^{N_2} \leq u_{m,k}^{N_1} + d_{m,k}^\uparrow.$$

When a neuron's tightened interval lies strictly in $\mathbb{R}_{<0}$ or $\mathbb{R}_{\geq 0}$, the binary $a_{m,k}$ is fixed and removed from the MIP.

There are three methods to compute $[d_{m,k}^\downarrow, d_{m,k}^\uparrow]$, each producing tighter bounds than the previous. The safe combinations use one of these three, controlled by the `zono` and `n1Probe` flags.

4.2.1 Method 1: Interval arithmetic (baseline)

Used when: `zono=false` and `n1Probe=off`.

This is the default method. From Phase 1 we have N_1 's per-neuron bounds $(l_{m,k}^{N_1}, u_{m,k}^{N_1})$, so the post-ReLU range is $z_{m,k}^{N_1} \in [\max(0, l_{m,k}^{N_1}), \max(0, u_{m,k}^{N_1})]$.

Weight-drift difference. Because N_1 and N_2 share the same architecture, every neuron (m, k) in N_2 has a corresponding neuron in N_1 . The *difference* in their pre-activations, $z_{m,k}^{N_2} - z_{m,k}^{N_1}$, depends only on the weight difference $\Delta W = W_2 - W_1$ (the “weight drift” from training). Sound intervals on this difference are computed layer by layer using interval arithmetic.

Affine layers. Let $\Delta w_{m,k} = w_{m,k}^2 - w_{m,k}^1$ and $\Delta b_{m,k} = b_{m,k}^2 - b_{m,k}^1$ be the weight and bias differences at layer m . From Phase 1 we have N_1 ’s per-neuron bounds $(l_{m,k}^{N_1}, u_{m,k}^{N_1})$ for every ReLU neuron. Since the post-ReLU output is $z_{m,k} = \max(0, \hat{z}_{m,k})$, the post-ReLU range for each neuron k in layer $m-1$ is $z_{m-1,k}^{N_1} \in [\max(0, l_{m-1,k}^{N_1}), \max(0, u_{m-1,k}^{N_1})]$.

The pre-activation difference at neuron k of layer m decomposes as:

$$\hat{z}_{m,k}^{N_2} - \hat{z}_{m,k}^{N_1} = \underbrace{\sum_{k'} w_{m,k,k'}^2 (z_{m-1,k'}^{N_2} - z_{m-1,k'}^{N_1})}_{\text{propagated diff from previous layer}} + \underbrace{\sum_{k'} \Delta w_{m,k,k'} \cdot z_{m-1,k'}^{N_1} + \Delta b_{m,k}}_{\text{direct weight-drift contribution}}.$$

Interval arithmetic bounds each term per output neuron k :

$$[d_{m,k}^\downarrow, d_{m,k}^\uparrow] = \sum_{k'} w_{m,k,k'}^2 [d_{m-1,k'}^\downarrow, d_{m-1,k'}^\uparrow] + \sum_{k'} \Delta w_{m,k,k'} [\max(0, l_{m-1,k'}^{N_1}), \max(0, u_{m-1,k'}^{N_1})] + \Delta b_{m,k},$$

ReLU layers. The ReLU difference satisfies $\sigma(z + d) - \sigma(z) \in [\min(0, d^\downarrow), \max(0, d^\uparrow)]$, so the diff interval is clipped: $d^\uparrow \leftarrow \max(0, d^\uparrow)$ and $d^\downarrow \leftarrow -\max(0, -d^\downarrow)$.

The final output at each layer is a sound interval:

$$d_{m,k}^\downarrow \leq z_{m,k}^{N_2} - z_{m,k}^{N_1} \leq d_{m,k}^\uparrow.$$

These are then combined with N_1 ’s known bounds to get N_2 bounds:

$$l_{m,k}^{N_2} \geq l_{m,k}^{N_1} + d_{m,k}^\downarrow, \quad u_{m,k}^{N_2} \leq u_{m,k}^{N_1} + d_{m,k}^\uparrow.$$

This method is fast but loose: it treats every neuron as independent, losing all correlations between neurons in the same layer.

4.2.2 Method 2: Zonotope arithmetic (Source A + B)

Used when: `zono=true`. **Flag:** `--adv_std_zono_bounds`.

This method computes the same $[d_{m,k}^\downarrow, d_{m,k}^\uparrow]$ as Method 1, but using *zonotope arithmetic* instead of interval arithmetic. A zonotope tracks correlations between neurons by representing each neuron’s value as a linear combination of shared noise variables, producing tighter bounds. Additionally, a second pass (Source B) independently bounds N_2 ’s absolute activations and intersects the result with the diff-based bounds.

Source A — Zonotope diff bounds. Each neuron’s difference is represented as a zonotope — a centre plus generators sharing noise variables $\epsilon_j \in [-1, 1]$ across all neurons in the layer. We store a centre vector $\mathbf{c}_m \in \mathbb{R}^K$ and a generator matrix $G_m \in \mathbb{R}^{K \times p}$ (rows = K neurons, columns = p generators, where p is the number of columns accumulated so far — initially $p = K_0$ at the input layer, and each affine layer adds K' columns while each ReLU adds one column per split neuron):

$$d_{m,k} = c_{m,k} + \sum_j g_{m,k,j} \epsilon_j, \quad d_{m,k}^\downarrow = c_{m,k} - \sum_j |g_{m,k,j}|, \quad d_{m,k}^\uparrow = c_{m,k} + \sum_j |g_{m,k,j}|.$$

Shared ϵ_j variables encode correlations: interval arithmetic is the special case where G_m is diagonal (no correlations).

Initialisation. At the input layer, N_1 and N_2 receive the same input, so the diff is zero: $\mathbf{c}_0 = \mathbf{0}$, $G_0 = \mathbf{0}_{K_0 \times 0}$ (no generators yet, $p = 0$). The first affine update creates the initial generators via G'_{indep} .

Affine layer update. Let $K' =$ neurons in layer $m-1$, $K =$ neurons in layer m :

$$c_{m,k} = \sum_{k'} w_{m,k,k'}^2 c_{m-1,k'} + \sum_{k'} \Delta w_{m,k,k'} \cdot \text{mid}_{k'} + \Delta b_{m,k} \quad (8)$$

$$\underbrace{G'_{\text{corr}}}_{K \times p} = \underbrace{W_2}_{K \times K'} \underbrace{G_{m-1}}_{K' \times p} \quad (9)$$

$$\underbrace{G'_{\text{indep}}}_{K \times K'} = \underbrace{\Delta W}_{K \times K'} \cdot \underbrace{\text{diag}(\text{rad}_1, \dots, \text{rad}_{K'})}_{K' \times K'} \quad (10)$$

$$\underbrace{G_m}_{K \times (p+K')} = \left[\underbrace{G'_{\text{corr}}}_{K \times p} \mid \underbrace{G'_{\text{indep}}}_{K \times K'} \right] \quad (11)$$

where $\text{mid}_{k'}$ and $\text{rad}_{k'}$ are the midpoint and half-width of N_1 's post-ReLU interval at neuron k' : $\text{mid}_{k'} = \frac{1}{2}(\max(0, u_{m-1,k'}^{N_1}) + \max(0, l_{m-1,k'}^{N_1}))$, $\text{rad}_{k'} = \frac{1}{2}(\max(0, u_{m-1,k'}^{N_1}) - \max(0, l_{m-1,k'}^{N_1}))$.

After the affine update, the column count grows from p to $p + K'$.

ReLU layer update (DeepZ). Compute hull per neuron: $l_k = c_{m,k} - \sum_j |g_{m,k,j}|$, $u_k = c_{m,k} + \sum_j |g_{m,k,j}|$.

- $u_k \leq 0$: $c_{m,k} \leftarrow 0$, $g_{m,k,\cdot} \leftarrow 0$.
- $l_k \geq 0$: unchanged.
- Split ($l_k < 0 < u_k$): $\lambda_k = \frac{u_k}{u_k - l_k}$, $\mu_k = -\frac{u_k \cdot l_k}{2(u_k - l_k)}$,

$$c_{m,k} \leftarrow \lambda_k c_{m,k} + \mu_k, \quad g_{m,k,\cdot} \leftarrow \lambda_k g_{m,k,\cdot},$$

append column to G_m : value μ_k at row k , zero elsewhere.

After the ReLU update, $p \leftarrow p + |\{k : l_k < 0 < u_k\}|$ (one new column per split neuron).

So for a network with L affine layers of widths $K_1, \dots, K_L$ and s_m split neurons at ReLU layer m :

$$p_{\text{final}} = K_0 + \sum_{m=1}^L (K_{m-1} + s_m).$$

Source B — N1-tightened absolute N2 zonotope. A separate zonotope propagates N_2 's *absolute* pre-activations (not the diff) through N_2 alone, using the same zonotope machinery as Source A but applied to N_2 's own values. Initialise from the full input range:

$$\mathbf{c}_0^{\text{B}} = \frac{1}{2} \mathbf{1}, \quad G_0^{\text{B}} = \text{diag}(\frac{1}{2} + \varepsilon, \dots, \frac{1}{2} + \varepsilon) \in \mathbb{R}^{n \times n},$$

covering $v_{\text{in}} \in [0, 1]^n$ with perturbation ε . Propagate through N_2 's affine layers ($c \leftarrow W_2 c + b_2$, $G \leftarrow W_2 G$) and DeepZ ReLU relaxations exactly as in Source A. At each ReLU layer, the zonotope hull gives per-neuron bounds:

$$u_{m,k}^{\text{hull}} = c_{m,k}^{\text{B}} + \sum_j |g_{m,k,j}^{\text{B}}|, \quad l_{m,k}^{\text{hull}} = c_{m,k}^{\text{B}} - \sum_j |g_{m,k,j}^{\text{B}}|.$$

These are then *intersected* with Source A’s diff-based bounds:

$$u_{m,k}^{N_2} = \min(u_{m,k}^{\text{hull}}, u_{m,k}^{N_1} + d_{m,k}^{\uparrow}), \quad l_{m,k}^{N_2} = \max(l_{m,k}^{\text{hull}}, l_{m,k}^{N_1} + d_{m,k}^{\downarrow}). \quad (12)$$

The first term in each min/max comes from Source B’s own zonotope pass on N_2 ; the second term comes from Source A’s diff bounds applied to N_1 ’s known bounds (the same formula as in the Goal paragraph above). Taking the tighter of the two at each neuron is sound because both are valid over-approximations.

The DeepZ ReLU relaxation is then applied using these tightened bounds, so Source B’s zonotope passes through a narrower “tube” at each layer.

4.2.3 Method 3: N1 Probe LP — additional tightening on top of Method 1 or 2 (Source C)

Used when: n1Probe=lp. **Flag:** --adv_std_n1_probe. **Perturbation support:** ℓ_∞ , brightness, contrast, patch, occlusion, translation. (Rotation is not yet supported; Source C is silently skipped for unsupported types.)

Method 3 is **not an alternative** to Methods 1/2 — it runs *after* them and further tightens the bounds they produced.

What the probe LP is. The probe LP is a **separate, temporary model** built from scratch — it is *not* the N_2 MIP. It contains:

- Shared input variables v_{in}, v_{x_0} (linked by perturbation-specific constraints).
- A complete forward pass through N_1 : affine layers (weights W_1 , biases b_1) with triangle-relaxed ReLUs.
- A complete forward pass through N_2 : affine layers (weights W_2 , biases b_2) with triangle-relaxed ReLUs.
- **No binary variables at all** — every ReLU in both networks uses the triangle LP relaxation (Eqs. (5)–(7)), so the entire model is a pure LP.

The key property is that N_1 and N_2 share the same input variables. When we optimize over this LP, the solver can reason about what inputs are feasible given the constraints of *both* networks simultaneously — this is what “cross-network correlations” means, and it is something that interval arithmetic and zonotopes (which process each network independently) cannot capture.

How it computes bounds. For each N_2 pre-activation neuron (m, k) : temporarily set the LP’s objective to $\hat{z}_{m,k}^{N_2}$, solve min and max, record the bounds, then change the objective to the next neuron. The bounds from Methods 1/2 are used as the triangle-relaxation intervals for N_2 ’s ReLUs inside this LP (tighter input bounds make the LP tighter).

LP construction. The LP encodes both N_1 and N_2 as triangle-relaxed forward passes sharing the same input variables v_{in} (clean) and v_{x_0} (perturbed). The only part that differs by perturbation type is the constraint linking v_{in} and v_{x_0} :

Perturbation	Constraint on (v_{in}, v_{x_0})
ℓ_∞	$v_{x_0} = v_{\text{in}} + v_e, \quad v_e \in [-\varepsilon, \varepsilon]^n$
brightness	$v_{x_0} = v_{\text{in}} + e, \quad e \in [0, \varepsilon]$ (scalar)
contrast	$v_{x_0,i} \in [v_{\text{in},i}, (1+\varepsilon)v_{\text{in},i}]$ (LP over-approx. of $e \cdot v_{\text{in}}$)
patch	$v_{x_0,i} = v_{\text{in},i} \pm \varepsilon$ on patch, $v_{x_0,i} = v_{\text{in},i}$ elsewhere
occlusion	$v_{x_0,i} = 0$ on occluded pixels, $v_{x_0,i} = v_{\text{in},i}$ elsewhere
translation	$v_{x_0} = \text{shifted } v_{\text{in}} \text{ with zero-padded borders}$

All constraints are linear (or, for contrast, a sound LP over-approximation of the bilinear $e \cdot v_{\text{in}}$), so the probe LP remains a pure LP for every supported perturbation type.

For ℓ_∞ the input variables are:

$$\begin{aligned} v_{\text{in}} &\in [0, 1]^n, & v_e &\in [-\varepsilon, \varepsilon]^n, & v_{x_0} &\in [0, 1]^n, \\ v_{x_0} &= v_{\text{in}} + v_e. \end{aligned} \tag{13}$$

Each ReLU neuron in both N_1 and N_2 is encoded using the triangle LP relaxation (Eqs. (5)–(7) from Section 3), making the entire model a pure LP (no binaries). The N_2 neuron bounds used in these triangle relaxations come from Method 1 or 2 (whichever ran first).

Optimisation-Based Bound Tightening (OBBT). For each N_2 pre-activation neuron (m, k) in both the original and perturbed network copies, solve two LPs:

$$l_{m,k}^{\text{probe}} = \min_{\mathcal{P}_{\text{LP}}} \hat{z}_{m,k}^{N_2}, \quad u_{m,k}^{\text{probe}} = \max_{\mathcal{P}_{\text{LP}}} \hat{z}_{m,k}^{N_2}, \tag{14}$$

where $\mathcal{P}_{\text{LP}}$ is the feasible polytope of the joint N_1+N_2 triangle-relaxed LP. Each neuron requires two LP solves (min and max), so the total cost is $2 \times (\text{number of } N_2 \text{ ReLU neurons}) \times 2$ (for org and pert copies).

Soundness of Method 3. The probe LP’s feasible set $\mathcal{P}_{\text{LP}}$ is an over-approximation of the true (input, N_1 -state, N_2 -state) feasible set: every true feasible point satisfies the triangle relaxation at each neuron, so $\mathcal{F}_{\text{true}} \subseteq \mathcal{P}_{\text{LP}}$. Therefore $l_{m,k}^{\text{probe}} \leq \min_{\mathcal{F}_{\text{true}}} \hat{z}_{m,k}^{N_2}$ and $u_{m,k}^{\text{probe}} \geq \max_{\mathcal{F}_{\text{true}}} \hat{z}_{m,k}^{N_2}$: the probe bounds are valid (possibly loose) per-neuron intervals. These bounds are then used only to tighten the big- M constants in N_2 ’s MIP encoding (via the final bound integration in Section 4.2.4). Since every tightened bound is satisfied by every $x \in \mathcal{F}_2$, the δ_{exact} -attaining point x^* remains feasible and the optimal $\delta_{N_2}^*$ is unchanged regardless of whether Method 3 is enabled. For contrast perturbation, the LP over-approximation of the bilinear $e \cdot v_{\text{in}}$ is looser than the exact constraint, but soundness is preserved: the LP feasible set is still a superset of the true feasible set.

4.2.4 Final bound integration

When N_2 ’s MIP is built, each ReLU neuron’s bounds are the tightest (intersection) of all available sources:

$$u_{m,k}^{N_2} \leftarrow \min(u_{m,k}^{\text{IA}}, u_{m,k}^{\text{B}}, u_{m,k}^{\text{probe,org}}, u_{m,k}^{\text{probe,pert}}), \tag{15}$$

$$l_{m,k}^{N_2} \leftarrow \max(l_{m,k}^{\text{IA}}, l_{m,k}^{\text{B}}, l_{m,k}^{\text{probe,org}}, l_{m,k}^{\text{probe,pert}}), \tag{16}$$

where IA denotes the Method 1 or 2 bounds (from $l_{m,k}^{N_1} + d_{m,k}^\downarrow$ and $u_{m,k}^{N_1} + d_{m,k}^\uparrow$), B denotes Source B (only when `zono=true`), and the two probe terms come from the OBBT (only when `n1Probe=lp`). Terms from disabled methods are simply omitted from the min/max. When the final interval is single-signed ($u_{m,k}^{N_2} \leq 0$ or $l_{m,k}^{N_2} \geq 0$), the associated binary is eliminated from the MIP entirely.

Soundness. All three methods produce sound over-approximations:

- Method 1 (interval arithmetic): standard interval arithmetic rules over-approximate the true range at each layer.
- Method 2 (zonotope): zonotope arithmetic is a sound abstract interpretation; Source B’s intersection of two sound over-approximations is itself sound.

- Method 3 (N1 probe): the triangle LP relaxation is a convex superset of the true feasible region, so OBBT over it yields valid bounds.

Every resulting bound is satisfied by every $x \in \mathcal{F}_2$, so x^* remains feasible and δ_{exact} is preserved.

Relationship to perturbed interval constraints. In addition to the bound-tightening described above, the N_2 MIP also receives *perturbed interval constraints* — these are added *after* the MIP is built and are **independent of Technique 4**. They are always enabled in the safe combinations (`--use_perturbed_intervals=true`) and operate as follows.

The VHAGaR MIP for a single network contains two copies: one for the clean input v_{in} (the “org” copy) and one for the perturbed input $v_{x_0} = v_{\text{in}} + v_e$ (the “perturbation” copy). Because both copies share the same network weights and their inputs differ by at most $v_e \in [-\varepsilon, \varepsilon]^n$, the post-ReLU activations of the two copies cannot differ by much. Perturbed interval constraints make this explicit: for each ReLU neuron (m, k) , compute sound intervals $[\Delta_{m,k}^\downarrow, \Delta_{m,k}^\uparrow]$ on the difference $z_{\text{pert}}^+(m, k) - z_{\text{org}}^+(m, k)$ by propagating $[-\varepsilon, \varepsilon]$ through the network using interval arithmetic. Then add two linking constraints directly to the MIP:

$$z_{\text{pert}}^+(m, k) \leq z_{\text{org}}^+(m, k) + \Delta_{m,k}^\uparrow, \quad (17)$$

$$z_{\text{pert}}^+(m, k) \geq z_{\text{org}}^+(m, k) + \Delta_{m,k}^\downarrow. \quad (18)$$

These constraints couple the org and perturbation copies, preventing them from diverging beyond what the perturbation bound allows. They are derived from interval arithmetic (sound), so they never exclude feasible points.

Summary of what operates where:

- **Technique 4** (bound tightening, Sources A/B/C): runs *before* MIP construction. Output: tighter per-neuron intervals $\rightarrow$ smaller big- M constants, fewer binaries.
- **Perturbed interval constraints**: runs *after* MIP construction. Output: additional linear constraints linking the org and perturbation copies of N_2 .
- Both are always active in the safe combinations and complement each other.

4.3 Technique 5 — Variable Hints (varHint)

Flag: `--adv_std_var_hint` $\in \{\text{false}, \text{true}\}$. **Gurobi attributes:** `VarHintVal`, `VarHintPri`.

Context: what Gurobi variable hints are. Gurobi exposes two per-variable attributes for *soft guidance*:

- **VarHintVal**: a suggested value for the variable (e.g. 0 or 1 for a binary). Gurobi uses this inside its internal heuristics — RINS (Relaxation Induced Neighbourhood Search), diving, and rounding — as a preferred direction when these heuristics need to fix a fractional variable. It is *not* a constraint: if the LP relaxation at some node disagrees with the hint, Gurobi is free to ignore it entirely.
- **VarHintPri**: an integer priority (≥ 0) that tells Gurobi how much to trust the hint. Higher priority means Gurobi weighs the hint more heavily in branching-direction decisions. A priority of 0 means “use the hint only if nothing else is available.”

The key property is that hints are *soft*: a wrong hint causes some wasted effort (Gurobi explores a suboptimal direction first) but can never change the final answer or make the solver return an incorrect optimum.

Idea. After solving N_1 , every binary variable a_i (the ReLU activation indicators) has a known optimal value $\text{val}_i^{N_1} \in \{0, 1\}$. Because N_2 is structurally identical to N_1 with only slightly different weights, most ReLU neurons settle to the same on/off state in both networks. We tell Gurobi: “for each binary in N_2 , try the value that was optimal in N_1 first.”

Variable matching. Because N_1 and N_2 share the same architecture, every binary activation variable in N_2 ’s MIP has a name-identical counterpart in N_1 ’s MIP. The matching is by *exact variable name* (e.g. `orga_layerCount1_neuronCount3.1.2`). Only binary variables are hinted; continuous variables (pre-/post-activations) are skipped.

Setting VarHintVal. For each binary a_i in N_2 whose name matches an N_1 variable:

$$\text{VarHintVal}(a_i) \leftarrow \text{val}_i^{N_1},$$

where $\text{val}_i^{N_1} \in \{0, 1\}$ is the optimal value of the matching binary in N_1 ’s solved MIP. This tells Gurobi “the same neuron’s activation was $\text{val}_i^{N_1}$ in N_1 ; try that value first in N_2 .”

Setting VarHintPri. The priority controls how strongly Gurobi trusts the hint. In the safe combinations, a uniform low confidence is assigned:

$$\text{VarHintPri}(a_i) \leftarrow 10 \quad \forall i.$$

A priority of 10 is low enough that Gurobi treats the hints as weak suggestions: it will follow them when it has no strong reason not to (e.g. during early heuristic rounds), but will override them whenever its own LP-based inference points in a different direction.

Why not submit the full N_1 solution as a candidate? Gurobi also supports **Start** (MIP Start), which submits a complete candidate solution that Gurobi attempts to *repair* into an N_2 -feasible point. This is a harder commitment: if the N_1 solution is far from N_2 -feasible, Gurobi wastes effort trying to repair it and may fail. Variable hints avoid this problem: each variable is nudged independently, and Gurobi can ignore any individual hint that does not work out. In practice, MIP Start was found to be harmful in all tested combinations (all LOSER), while variable hints contribute to 7 of the 17 safe combinations.

Effect on the solve. With hints active, the first few rounds of Gurobi’s feasibility heuristics tend to find an incumbent faster: the hints pull the heuristics toward N_1 -like binary patterns, which are already close to N_2 -optimal for most neurons. Having an early incumbent tightens Gurobi’s **Cutoff** (the objective threshold below which sub-trees are pruned), which accelerates the subsequent branch-and-bound search.

Soundness. `VarHintVal` and `VarHintPri` are advisory attributes: no constraint is added, no coefficient is changed, no variable bound is modified. The MIP feasible set is unchanged: $\mathcal{F}_2^{(\{5\})} = \mathcal{F}_2$ and δ_{exact} is preserved. Wrong hints may waste some effort but cannot produce a wrong answer.

4.4 Technique 6 — N2 Triangle Relaxation (**relaxT**)

Flag: `--adv_std_n2_relax_threshold` $\in \{0.01, 0.1\}$ (in safe combos). **Model modification:** replaces selected N2 binary variables with LP relaxations.

Idea. For each ReLU neuron in N_2 (and its perturbed copy N_2'), look up N_1 's solved bounds at the *same* neuron (matched by (m, k) layer/neuron key) and compute the *triangle gap area*. If this gap is below the threshold τ , the binary activation variable is replaced with a pure LP triangle relaxation (no binary).

Triangle gap area. For a ReLU neuron with pre-activation bounds $[l, u]$ from N_1 , the gap area measures how well the convex triangle relaxation approximates the exact ReLU:

$$\text{gap}(l, u) = \begin{cases} 0 & \text{if } l \geq 0 \text{ or } u \leq 0 \quad (\text{stable neuron}), \\ \frac{u \cdot |l|}{2(u - l)} & \text{otherwise (split neuron)}. \end{cases} \quad (19)$$

Geometrically, this is the area of the triangle between the exact ReLU function $\max(0, \hat{z})$ and its upper envelope $\frac{u}{u-l}(\hat{z} - l)$ on the interval $[l, 0]$. A small gap means the LP relaxation is already a tight approximation of the ReLU, so encoding the binary offers little benefit.

Decision rule. For each ReLU neuron (m, k) during N_2 's MIP construction:

1. Look up N_1 's tight bounds $(u_{m,k}^{N_1}, l_{m,k}^{N_1})$ from the bound-tightening dictionary (populated by Technique 4).
2. Compute $\text{gap}_{m,k} = \text{gap}(l_{m,k}^{N_1}, u_{m,k}^{N_1})$.
3. If $\text{gap}_{m,k} \leq \tau$: **relax** (use LP triangle). Otherwise: **encode exactly** (use big- M with binary).

This is applied independently to both the original copy $N_2(v_{\text{in}})$ and the perturbed copy $N_2(v_{x_0})$, tracked separately as $n_{\text{relaxed}}^{\text{org}}$ and $n_{\text{relaxed}}^{\text{pert}}$.

What happens at each neuron. When $\text{gap}_{m,k} > \tau$, the neuron is encoded exactly using the standard big- M formulation with a binary variable (Eqs. (1)–(4) in Section 3). When $\text{gap}_{m,k} \leq \tau$, the binary a is **not created** and the four big- M constraints are replaced by the three-constraint triangle LP relaxation (Eqs. (5)–(7) in Section 3). The feasible set of z^+ becomes the convex hull of the ReLU graph on $[l, u]$ — a triangle in the $(\hat{z}, z^+)$ plane with no binary variable.

Effect. Each relaxed neuron removes one binary variable and one constraint from the MIP. When $\tau = 0.01$, only neurons where N_1 's LP triangle is already nearly exact are relaxed (conservative). When $\tau = 0.1$, more neurons are relaxed (aggressive). The reported δ satisfies:

$$\delta_{\text{relaxed}} \geq \delta_{\text{exact}},$$

since the relaxed feasible set is a superset of the exact one.

Threshold values. The safe combinations use $\tau \in \{0.01, 0.1\}$. Values $\tau \geq 0.5$ are not safe: they relax too many neurons, and the resulting LP relaxation can be so loose that Gurobi's dual bound converges slowly, causing catastrophic slowdowns ($0.053\times$) on some instances.

Soundness. The triangle LP relaxation (Eqs. (5)–(7)) is a *convex relaxation* of the exact big- M encoding (Eqs. (1)–(4)): every point satisfying the exact constraints also satisfies the relaxed constraints. Therefore $\mathcal{F}_2 \subseteq \mathcal{F}_2^{\{6\}}$ and the δ_{exact} -attaining x^* remains feasible. The maximum over the relaxed set can only be $\geq \delta_{\text{exact}}$, never below it. The bound is sound even when loose.

5 Safe-to-Use Combinations

Table 1 lists all 17 combinations empirically validated as “GENERAL — safe to use”: they have at least one stable-WIN seed, no stable-LOSE seed, and worst-case slowdown $> 0.8\times$. Testing was performed on MNIST with architectures `cnn1` and `3x10`, $c_{\text{targets}} \in \{3, 4, 5\}$, and 5 Gurobi seeds. Bound tightening is always enabled and relax threshold is always in $\{0.01, 0.1\}$; these are omitted from the table columns for brevity.

#	bp	varHint	zono	n1Probe	τ	gm.all	max_slow
1	off	no	yes	lp	0.1	1.504 $\times$	0.804 $\times$
2	bounds	no	yes	off	0.01	1.490 $\times$	0.841 $\times$
3	off	no	no	lp	0.1	1.485 $\times$	0.814 $\times$
4	bounds	no	no	lp	0.01	1.479 $\times$	0.826 $\times$
5	off	no	yes	off	0.01	1.469 $\times$	0.846 $\times$
6	bounds	no	yes	lp	0.1	1.465 $\times$	0.846 $\times$
7	bounds	no	no	off	0.1	1.449 $\times$	0.846 $\times$
8	off	no	no	off	0.1	1.448 $\times$	0.841 $\times$
9	off	no	yes	lp	0.01	1.424 $\times$	0.836 $\times$
10	off	yes	yes	off	0.01	1.403 $\times$	0.808 $\times$
11	off	no	no	off	0.01	1.398 $\times$	0.845 $\times$
12	bounds	yes	yes	lp	0.1	1.393 $\times$	0.812 $\times$
13	bounds	yes	no	off	0.1	1.379 $\times$	0.826 $\times$
14	bounds	yes	yes	off	0.01	1.361 $\times$	0.817 $\times$
15	bounds	yes	yes	off	0.1	1.357 $\times$	0.822 $\times$
16	off	yes	yes	off	0.1	1.343 $\times$	0.814 $\times$
17	off	yes	yes	lp	0.01	1.328 $\times$	0.817 $\times$

Table 1: All 17 safe-to-use `advstd` combinations, ranked by geometric mean speedup over standard mode. All have `boundTight=true`, `mipStart=false`, `lpBasis=false`. The τ column is `--adv_std_n2_relax_threshold`.

6 Algorithms

All 17 safe combinations share the same two-phase structure (Algorithm 1 and Algorithm 2). They differ only in which optional sub-steps are enabled, as indicated by the Boolean flags `BP`, `ZONO`, `PROBE`, `VARHINT`, and the threshold τ .

Algorithm 1 Phase 1: Solve N_1 and extract state (shared by all safe combos)

Require: Network N_1 , perturbation (P, ε) , class pairs (c_s, c_t) , timeout T

Ensure: State directory $\mathcal{S}$ on disk

```
1: for each class pair  $(c_s, c_t)$  do
2:    $x_{\text{pgd}} \leftarrow \text{HyperAttack}(N_1, c_s, c_t, P, \varepsilon)$  {PGD warm-start}
3:   Build MIP  $\mathcal{M}_1$  for  $N_1$  with standard encoding
4:   Apply PGD hints to  $\mathcal{M}_1$ 
5:   Add VHAGaR dependency constraints to  $\mathcal{M}_1$ 
6:   Add perturbed interval constraints to  $\mathcal{M}_1$ 
7:   optimize! $(\mathcal{M}_1, T)$ 
8:   Extract primal solution  $(\text{var}_i, \text{val}_i)_i$ 
9:   Extract neuron bounds  $(l_{m,k}, u_{m,k})_{\forall m,k}$  from layers_info
10:  Serialise to  $\mathcal{S}/(c_s, c_t)$ 
11: end for
```

Algorithm 2 Phase 2: Solve N_2 with advstd techniques (parameterised)

Require: Networks N_1, N_2 , perturbation (P, ε) , class pairs, timeout T , N1 state $\mathcal{S}$

Require: Flags: BP $\in \{\text{off}, \text{bounds}\}$, ZONO $\in \{\text{false}, \text{true}\}$, PROBE $\in \{\text{off}, \text{lp}\}$, VARHINT $\in \{\text{false}, \text{true}\}$, $\tau \in \{0.01, 0.1\}$

Ensure: Global robustness bound $\delta_{N_2}^*$ for each (c_s, c_t)

```

1: // Technique 4: Precompute N2 bounds from N1 (once, before the loop)
2: Load  $N_1$  diff bounds from  $\mathcal{S}$ 
3: if ZONO then
4:   Compute zonotope diff bounds (Source A) between  $N_1$  and  $N_2$ 
5:   Compute N1-tightened absolute N2 zonotope (Source B), intersect with Source A
6: else
7:   Compute interval-arithmetic diff bounds between  $N_1$  and  $N_2$ 
8: end if
9: if PROBE = lp then
10:   Build joint  $N_1+N_2$  LP with triangle-relaxed ReLUs (Source C)
11:   Run per-neuron OBBT to tighten N2 pre-activation bounds
12: end if
13: for each class pair  $(c_s, c_t)$  do
14:   // Skip if result already exists for this pair
15:   if result file contains  $(c_s, c_t)$  then
16:     continue
17:   end if
18:   Load  $N_1$  state from  $\mathcal{S}/(c_s, c_t)$ : primal  $(\text{var}_i, \text{val}_i)$ , bounds  $(l_{m,k}, u_{m,k})$ 
19:   // Technique 4: Set N1 neuron bounds for N2 encoding
20:   Inject  $(l_{m,k}, u_{m,k})$  as initial N2 bounds {fixes stable ReLUs, tightens big- $M$ }
21:   // Technique 6: Replace near-stable N2 binaries with LP relaxations
22:   for each ReLU neuron  $(m, k)$  in N2 and N2' do
23:     if triangle gap area of  $N_1$ 's interval at  $(m, k) \leq \tau$  then
24:       Replace binary  $a_{m,k}$  with triangle LP relaxation
25:     end if
26:   end for
27:   Build MIP  $\mathcal{M}_2$  for  $N_2$  {with tightened bounds and relaxed neurons}
28:    $x_{\text{pgd}} \leftarrow \text{HyperAttack}(N_2, c_s, c_t, P, \varepsilon)$ 
29:   Apply PGD hints to  $\mathcal{M}_2$ 
30:   if BP = bounds then
31:     // Technique 2: Branch priorities from N1 bound gaps
32:     for each binary  $a_{m,k}$  in  $\mathcal{M}_2$  do
33:        $g \leftarrow |u_{m,k} - l_{m,k}|$ 
34:       Set BranchPriority( $a_{m,k}$ )  $\leftarrow \begin{cases} 10 & g < 1 \\ 5 & 1 \leq g < 5 \\ 1 & \text{else} \end{cases}$ 
35:     end for
36:   end if
37:   if VARHINT then
38:     // Technique 5: Soft variable hints from N1 solution
39:     for each binary  $a_i$  in  $\mathcal{M}_2$  with matching  $N_1$  variable do
40:       Set VarHintVal( $a_i$ )  $\leftarrow \text{val}_i^{N_1}$ 
41:       Set VarHintPri( $a_i$ )  $\leftarrow 10$  {uniform low confidence}
42:     end for
43:   end if
44:   Add VHAGaR dependency constraints to  $\mathcal{M}_2$ 
45:   Add perturbed interval constraints to  $\mathcal{M}_2$ 
46:    $\text{minimize } (\mathcal{M}_2, T)$ 

```

6.1 Instantiation for Each Safe Combination

Table 2 maps each safe combination to its flag settings in Algorithm 2. To reproduce any combination, set the five flags accordingly; all other aspects of the algorithm are identical.

#	BP	VARHINT	ZONO	PROBE	τ
1	off	false	true	lp	0.1
2	bounds	false	true	off	0.01
3	off	false	false	lp	0.1
4	bounds	false	false	lp	0.01
5	off	false	true	off	0.01
6	bounds	false	true	lp	0.1
7	bounds	false	false	off	0.1
8	off	false	false	off	0.1
9	off	false	true	lp	0.01
10	off	true	true	off	0.01
11	off	false	false	off	0.01
12	bounds	true	true	lp	0.1
13	bounds	true	false	off	0.1
14	bounds	true	true	off	0.01
15	bounds	true	true	off	0.1
16	off	true	true	off	0.1
17	off	true	true	lp	0.01

Table 2: Flag instantiation for each safe combination (numbered as in Table 1). `boundTight` is always true and is not shown.

7 Summary of Effects

Table 3 summarises which Gurobi attribute each safe technique uses and whether it modifies the mathematical problem.

Technique	Flag (short)	Gurobi API	Modifies MIP?	N_1 state consumed
2. Branch Prio bounds	<code>bp=bounds</code>	<code>BranchPriority</code>	no	final neuron bounds
4. Bound Tightening	<code>boundTight</code>	(linear constr.)	yes [†]	final neuron bounds
4+. Zonotope Bounds	<code>zono</code>	(linear constr.)	yes [†]	diff bounds
4+. N1 Probe LP	<code>n1Probe=lp</code>	(linear constr.)	yes [†]	network weights
5. Variable Hints	<code>varHint</code>	<code>VarHintVal+VarHintPri</code>	no	primal solution
6. N2 Relaxation	<code>relaxT=τ</code>	(replaces binaries)	yes [‡]	N1 neuron bounds

Table 3: Summary of the safe advstd techniques. [†]Adds sound constraints derived from over-approximations; the optimum is preserved. [‡]Replaces binaries with LP relaxations; the reported δ is a sound upper bound ($\geq \delta_{\text{exact}}$).

8 Command-Line Interface

A minimal invocation using the top-ranked safe combination (#1) is:

```
julia run.jl --mode advanced_standard_n2 \
--dataset mnist --model_name cnn1 \
--model_path /path/to/N1/model.p \
```

```

--model_path2 /path/to/N2/model.p \
--perturbation_patch --perturbation_size "1,14,14,3" \
--ctag 1 --ct "4,5,6" --timeout 1800 \
--n1_state_dir /path/to/n1_state \
--adv_std_mip_start false \
--adv_std_branch_priorities off \
--adv_std_lp_basis false \
--adv_std_bound_tightening true \
--adv_std_zono_bounds true \
--adv_std_n1_probe lp \
--adv_std_n2_relax_threshold 0.1 \
--adv_std_var_hint false

```

The Python sweep script can run all 17 safe combinations automatically:

```

python3 run_relaxation_sweep.py --timeout 1800 \
--arch_models cnn1=/path/to/n1 3x10=/path/to/n1 \
--advanced_standard \
--sweep_adv_std_mip_start false \
--sweep_adv_std_lp_basis false \
--sweep_adv_std_bound_tightening true \
--sweep_adv_std_zono_bounds false true \
--sweep_adv_std_n1_probe off lp \
--sweep_adv_std_n2_relax_threshold 0.01 0.1 \
--sweep_adv_std_branch_priorities off bounds \
--sweep_adv_std_var_hint false true \
--advstd_safe_combos_only /path/to/ranking.csv \
--sweep_gurobi_seed 0 1 2 3 4 --ct 4,5,6

```

The `--advstd_safe_combos_only` flag filters the cross-product to only the 17 validated combinations, blocking known GENERAL/MIXED/LOSER combos while allowing untested new combinations through.